DATA ENGINEERING USING AWS DATA ANALYTICS

Section 2 : Set up local environment for AWS on Windows 10/11

Powershell

Command prompt in Windows. More powerful than the DOS prompt.
Ability to SSH to remote machines without any additional software.
Customize by right clicking on top -> properties.
Commands:

- ssh - to connect to secure shell
- dir - to list directory

Set up Ubuntu on Windows

Windows subsystem for Linux (WSL)

Powershell commands: (prompt starts with >)
wsl -l -v : to list the existing version
wsl : to connect to default wsl (* represents default)
wsl -d Ubuntu-20.04 : to connect to specific wsl
wsl.exe --list --online : to check the available distributions
wsl --install -d Ubuntu-20.04 : d is distribution
Then reboot / restart the system.

WSL CLI (Ubuntu shell) opens: (prompt start with $)
create username and password
uname -a : get OS details
wsl
ls -ltr : to see all files in the home folder
exit
dir

cd : change directory
pwd : present working directory

Set up Pyhton venv and pip in Ubuntu

python
python3 exit()
sudo apt-get install python3-venv
sudo apt update // to update list
sudo apt-get install python3-venv // run again to download updated list
sudo apt-get install python3-pip // to install pip
python3 -m venv demo-venv //create folder named demo-venv
ls -ltr //to see if the folder is created
rm -rf demo-venv/ //to remove the folder
pip install configparser

Set up AWS CLI on Ubuntu using pip

pip install awscli
aws (or type: exit -> wsl -> aws)
aws --version

Create AWS IAM user and download credentials

Log in to AWS console
Search for IAM
In the IAM dashboard, on the left pane, select Users
Click ‘add users’
Give username, programmatic access-access key. Give it admin access.Click ‘create user’. Download the csv file.

In the powershell wsl,
aws
aws s3 ls //list the s3 buckets available in the account
aws configure //create default profile. copy paste the downloaded access, secret key. Leave region, output format blank
ls -ltr .aws
cat .aws/credentials
aws s3 ls // run again to validate configuration

rm .aws/credentials //remove the file
aws configure --profile awsdemo //create a profile, it'll ask for access key
aws s3 ls --profile awsdemo

Create Python virtual environment

mkdir -p Projects/internal/data-on-aws //create directory
cd Projects/internal/data-on-aws // to access the directory
python3 -m venv de-venv //m is module. Create the virtual env with the name de-venv
ls -ltr de-venv (or) find de-venv
source de-venv/bin/activate //activate the env

Set up Boto3 in Python venv

Boto3 is a SDK.
pip install boto3
python //prompt changes to >>
import boto3 //to validate installation
s3_client = boto3.client('s3')
s3_client.list_buckets()

Set up Jupyter lab and validate boto3

pip install jupyterlab
jupyter lab //paste the URL given to the browser

In the Jupyter lab, open a notebook,
import boto3
s3_client = boto3.client('s3') //create object
type(s3_client)
buckets = s3_client.list_buckets( ) ['Buckets']
bucket = buckets[0]
bucket['Name']
bucket_names = [bucket['Name'] for bucket in buckets]
bucket_names

Section 3 : Set up local environment on MAC

Section 4 : Set up environment for practice using Cloud9

It is the official AWS IDE.

Set up Cloud9

Go to AWS management console, search for IAM
Go to IAM console.
On the left panel, click ‘Groups’
Create new group
Group name: ITVCloud9
Attach policy: Admin access
Then create group.

Click Users from the left panel.
Create user
Username: ITVCloud9User
Give programmatic access and AWS management access
Add ITVCloud9 to user
Copy the access URL to the browser. Then log in
Change password.

Do not create users using a root account, only an IAM account.

Search Cloud9, open Cloud9 console.
Click ‘create environment’.
Give name: awsdemo
In configure settings: ec2 direct access, t2.micro, linux2, after 30 min. Review and create.

Open the Cloud9 terminal.
Open new file: rename and save
Validate tools: git, docker ps -a, sudo systemctl status httpd, python (press tab twice), exit, python2, exit( ), java -version, javac -version,

Docker and AWS on Cloud9

aws s3 ls // to see if aws access is there
sudo systemctl status docker
docker ps //to see running containers

docker images
docker run hello-world //stops by itself after running
docker rmi hello-world //to remove

Cloud9 and EC2

Log in to ITV user.
Search Cloud9, click awsdemo, click EC2 instance.
Cloud9 is backed by EC2 instance.

Accessing web applications
Open the terminal, click preview on top.

sudo systemctl status httpd
sudo systemctl start httpd
telnet localhost 80
nc
sudo yum install telnet -y
telnet localhost 80
(or)
In aws console, instances, click instance, security, security groups, edit inbound rules.

Allocate and assign static IP

In the left pane, under Network and Security, select Elastic IPs.
Click 'Allocate elastic IP address'.
Once done, from the top pane, select Actions -> allocate elastic IP address -> click instance

Set up Jupyter lab on Cloud9 instance

Prerequisites Check: Validate Python3 availability: python (shows Python 2.7) and python3 (shows Python 3.6.9)

Setup Steps
1. Create Project Directory
mkdir delab
cd delab

2. Create Virtual Environment
python3 -m venv delab-venv

- Verify creation: find delab-venv or ls -ltr delab-venv/bin

3. Activate Virtual Environment
source delab-venv/bin/activate

4. Install Jupyter Lab
pip install jupyter lab

Running Jupyter Lab
Option 1: Direct Run (Foreground)
jupyter lab --ip 0.0.0.0

- Starts web server on port 8888
- Console remains occupied
- Token displayed in terminal

Option 2: Background Run
nohup jupyter lab --ip 0.0.0.0 &

- Runs in background, frees up terminal
- View token: tail nohup.out

Network Configuration
Configure Security Group

1. Go to EC2 instances in AWS console
2. Select your Cloud9 instance
3. Navigate to Security → Security Groups
4. Edit inbound rules
5. Add rule: Custom TCP, Port 8888, Source: My IP
6. Save rules

Access Jupyter Lab

- URL format: http://[public-ipv4-dns]:8888
- Enter token from terminal when prompted (token in terminal)
- After login, full Jupyter Lab interface is accessible

Using Jupyter Lab
Terminal Access

- Click terminal icon to open Ubuntu terminal
- Run standard Ubuntu commands

Creating Notebooks

- Click Python3 icon to create new notebook
- Rename notebook as needed (e.g., "getting_started")
- Write and execute Python code cells

Example Code

```
from platform import python_version
python_version()  # Shows Python 3.6.9
```

Key Points

- Must activate virtual environment before starting Jupyter Lab
- IP must be set to 0.0.0.0 for external access
- Port 8888 must be open in security group
- Token required for first login
- Use nohup for background operation if you need the terminal for other tasks

Open SSH port for Cloud9 EC2 instance

- Development environment uses AWS Cloud9 (IDE on EC2 instance)
- Elastic IP associated with EC2 instance for consistent access
- Terminal accessible via Cloud9 IDE, but SSH access provides alternative connection method

Accessing Terminal

Method 1: Through Cloud9 IDE

- Click terminal icon in IDE
- Launch terminal directly
- Run Linux commands

Method 2: SSH Connection

- Connect directly to EC2 instance using SSH
- Requires port 22 configuration

Getting Public DNS/IP

1. Navigate to Cloud9 environment (e.g., "delabdemo")
2. Click on environment name
3. Scroll down and click "Go to instance"
4. Select the instance
5. Scroll down to find:
    - Public IPv4 address
    - Public IPv4 DNS

Testing Port 22 Connectivity

Test Command

bash
telnet [public-DNS] 22
Expected Results

- Port Open: Shows prompt immediately
- Port Closed: Connection hangs with no prompt

Exit Telnet
Ctrl +]
quit

Configuring Security Group for SSH
Steps to Open Port 22

1. Go to AWS console → EC2 instance
2. Navigate to Security tab
3. Click on the security group link
4. Click Edit inbound rules
5. Add/modify SSH rule (port 22)

Source IP Options
Option 1: My IP (Recommended for personal accounts)

- Restricts access to your current IP only
- More secure

Option 2: Anywhere-IPv4

- Allows access from any IP address
- Acceptable for personal learning environments
- NOT recommended for company AWS accounts

Option 3: SSH Bastion (For corporate environments)

- Use SSH bastion/jump host concept
- Required for company AWS accounts

6. Save rules

Verification After Configuration
bash
telnet [public-DNS] 22

- Should show prompt immediately if successful
- Exit: Ctrl +] then quit

Next Steps

- Configure SSH keys for authentication
- Connect from local machine to EC2 instance using SSH

Security Best Practices

Environment Type	Recommended Setting
Personal Learning	My IP or Anywhere-IPv4 acceptable
Company AWS Account	My IP only or SSH Bastion

Note: SSH key configuration required before actual SSH connection is possible.

Connect to Cloud9 EC2 instance using SSH

Step 1: Check EC2 Instance SSH Directory
bash
cd ~
pwd # Shows home directory
ls -ltr .ssh # Shows SSH files including authorized_keys

Step 2: Generate SSH Key Pair (On Local Mac/PC)
Works on: Mac Terminal, Windows PowerShell, or Git Bash
bash
ssh-keygen
When prompted:

- File path: ~/.ssh/itvc9demo (or your preferred name)
- Passphrase: Press Enter to skip (optional)

Result: Creates two files:

- itvc9demo (private key - keep on your local machine)
- itvc9demo.pub (public key - copy to server)

Step 3: Copy Public Key Contents
bash
cat ~/.ssh/itvc9demo.pub
Important: Select and copy from ssh-rsa to the very last character. Don't include extra characters or spaces.
Step 4: Add Public Key to EC2 authorized_keys
CRITICAL: Be extremely careful - corrupting authorized_keys can break Cloud9 access!

1. Open the file in Cloud9 terminal:

bash
vi ~/.ssh/authorized_keys

2. Navigate to end of file:
    - Press Shift + G (goes to last line)
    - Press $ (goes to end of line)

3. Enter append mode:
    - Press a (watch for "INSERT" mode indicator)
    - Press Enter to create new line
4. Paste public key:
    - Press Cmd + V (Mac) or Ctrl + V (Windows)
    - Verify the key pasted correctly
5. Save and exit:
    - Press Esc (exit insert mode)
    - Type :x and press Enter

Step 5: Verify Addition
bash
tail -1 ~/.ssh/authorized_keys
Should show your newly added public key.

Part 3: Connect via SSH
SSH Connection Command
bash
ssh -i ~/.ssh/itvc9demo ubuntu@[public-DNS]
Components:

- -i ~/.ssh/itvc9demo: Path to private key file
- ubuntu: Username (default for Cloud9 EC2 instances)
- [public-DNS]: Public DNS from EC2 console

First-Time Connection

- Prompt: "Are you sure you want to continue connecting?"
- Type: yes and press Enter
- Should connect successfully to EC2 instance

Troubleshooting

Issue	Solution
Connection refused	Verify port 22 is open in security group
Permission denied	Check private key path and authorized_keys content
Key format error	Ensure public key copied completely without extra spaces
Cloud9 access broken	Corrupted authorized_keys - may need AWS support

Security Reminders

- Never share your private key (itvc9demo)

- Only add public keys to authorized_keys (itvc9demo.pub)
- Back up authorized_keys before editing
- Use My IP restriction for company AWS accounts
- Test connection immediately after configuration

Quick Reference;

```
bash
# Generate key pair
ssh-keygen
# View public key
cat ~/.ssh/[keyname].pub
# Connect to EC2
ssh -i ~/.ssh/[keyname] ubuntu@[public-DNS]
# Verify authorized_keys
tail -1 ~/.ssh/authorized_keys
```